

VLA 真机后训练的数据生成与经验利用

PLD 与 RECAP 的机制级解读，及由开放问题导出的 HEARS-Buffer 方向

刘越（同济大学计算机学院 空间智能课题组）

2026-06

Contents

1 摘要与阅读导引	2
2 问题设定：为什么真机 VLA 后训练这么难	2
2.1 VLA 策略的形式	2
2.2 真机 RL 的形式化目标	2
2.3 为什么不能“直接对 VLA 做 RL”	2
3 支撑机制	3
3.1 离线 RL 的核心病理：OOD 动作的 Q 值高估	3
3.2 CQL $\rightarrow$ Cal-QL：保守价值学习与“校准”	3
3.3 RLPD：用四个最小设计在不加保守项的前提下稳住 off-policy 学习	4
3.4 AWR / AWAC：advantage-weighted regression（RECAP 的理论近亲）	4
3.5 Classifier-Free Guidance（CFG）	5
4 PLD：用残差 RL 做“部署对齐的数据生成”	5
4.1 残差策略的精确形式，以及为什么是残差而非 LoRA	5
4.2 Stage 1（Probe + Learn）：Cal-QL critic + 残差 actor	6
4.3 Stage 2（Probing）：PLD 真正的灵魂	6
4.4 Stage 3（Distill）：普通 SFT，但防遗忘机制在 Stage 2	7
4.5 实验关键数字与技术局限	7
5 RECAP：把 advantage 从 loss 通道挪到 conditioning 通道	7
5.1 三阶段，以及为什么每轮从 anchor 重训	8
5.2 从 0/1 标签到 step reward，到分布式 value	8
5.3 advantage、二值 indicator 与三类数据的不对称处理	9
5.4 策略训练：CFG-style 双支目标 + prefix 注入 + random drop	9
5.5 CFG 推理与 flow-matching 下界	10
5.6 为什么绕开了 PPO：AWR 视角与单调改进下界	10
5.7 迭代改进的微观机制： I_t 会翻转	10
5.8 实验关键数字	10
6 PLD 与 RECAP：机制级对比	11
7 由开放问题导出的研究方向：HEARS-Buffer	11
8 训练动力学：第 1 章诸多论断的理论支撑	12
9 关键文献	13

1 摘要与阅读导引

本报告从算法机制的角度，系统解读 VLA (Vision-Language-Action) 模型在真机后训练上的两条代表性自改进 (self-improving) 路线，并由它们共同的技术开放问题导出一个数据层研究方向。我们刻意不做“范式 A 做 X、范式 B 做 Y”式的清单对比，而是把每个方法的目标函数、算法步骤、设计推导与失败模式讲清楚。

两条主线的技术分野可以用一句话概括：

- **PLD** (Probe-Learn-Distill) 把“自改进”实现为一次 **off-policy 残差 RL 驱动的数据生成**：冻结 VLA 主干，训练一个轻量残差专家去接管 VLA 的失败状态，用一种“基策略探针 + 残差救场”的混合 rollout 生成“贴着 VLA 部署分布、且含纠错行为”的数据，最后用标准 SFT 把数据蒸馏回 VLA。RL 的产物是数据，专家训完即弃。
- **RECAP** ($\pi^*0.6$ 的训练配方) 把“自改进”实现为一次 **迭代式 offline RL**：用一个分布式 value function 把 episode 级 0/1 成功标签升级成 step 级 advantage，再把 advantage 当作条件输入 (conditioning) 而非损失项 (loss) 注入策略，靠 classifier-free guidance 式的联合训练实现 advantage 加权的纯监督学习。RL 的信号走 conditioning 通道，完全绕开 PPO。

阅读路径：第 1 章给出问题的形式化设定，并解释为什么“直接对 VLA 做 RL”在真机上不可行；第 2 章是支撑机制 (Cal-QL、RLPD、AWR/AWAC、CFG)，它们是理解 PLD/RECAP 的前提；第 3、4 章分别深读 PLD 与 RECAP；第 5 章做机制级对比；第 6 章从二者共同的技术开放问题导出 **HEARS-Buffer** 方向；第 7 章是 RL/SFT 训练动力学的理论补充（解释第 1 章诸多论断的“为什么”）。

2 问题设定：为什么真机 VLA 后训练这么难

2.1 VLA 策略的形式

被后训练的对象是一个语言条件的视觉-动作策略 $\pi_\theta(a | o, g)$ ： o 是部分观测（本体感知 + 多路 RGB）， g 是语言目标，输出 a 是 7-DoF 动作（6-DoF 末端位姿增量 + 1-DoF 连续夹爪）。现代 VLA 普遍用 **action chunking**——一次预测未来 H 步的动作块并开环执行若干步——以提升时序一致性与推理效率（PLD 真机用 chunk size 26、执行 horizon 15）。动作头有三种主流形式，它们决定了“能否对策略求 $\log \pi$ ”这一关键性质：

- **离散 token 头** (OpenVLA 风格)：动作离散化后自回归生成，有解析的 $\log \pi$ ，交叉熵训练；
- **扩散 / flow-matching 头** (π_0 风格)：动作由一个连续生成过程产生，没有解析的 $\log \pi$ ，只能用 score-matching / flow-matching 的 MSE 损失。

这第二类的“无解析似然”是后面理解 RECAP 为何要用 log-likelihood 下界、为何天然适配 CFG 的关键。

2.2 真机 RL 的形式化目标

把控制过程建模为目标条件 MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \rho, \rho_0, r, \gamma)$ ，真机任务的奖励几乎总是稀疏二元的：

$$r(s, a, g) = \mathbf{1}[d(\phi(s), g) \leq \varepsilon] \in \{0, 1\},$$

即只有目标相关表示 $\phi(s)$ 落入容差时才给 1。优化目标是标准折扣回报 $J(\pi) = \mathbb{E}[\sum_t \gamma^t r(s_t, a_t, g)]$ 。这个看似平凡的设定隐藏了真机后训练的全部困难。

2.3 为什么不能“直接对 VLA 做 RL”

把 SFT 与 RL 直接相加、或在 VLA 上跑 PPO，在真机上遇到四个叠加的困难——注意这不是“梯度偶尔大”，而是四个因素在机制上彼此放大：

1. **更新次数量级差**。一次 SFT 约 10^2 步梯度即收敛；真机 RL 要把稀疏信号 bootstrap 出有效策略，需 10^5 量级的更新（PLD 实测约 250k actor steps）。在共享参数上施加千倍量级的更新，是通用能力被覆盖的物理前提。
2. **梯度方向的高方差**。稀疏奖励下，一条 50 步轨迹只在末尾拿到 1 个标量，“哪一步对成功有贡献”完全靠 value 函数隐式 bootstrap——这本身是高方差的信用分配问题（见第 2.1 节的 OOD 高估病理）。
3. **冷启动的零信号**。SFT 成功率低于约 20% 时，前若干条 rollout 全失败、reward 恒为 0、策略梯度恒为 0；偶然的第一次成功又会带来一次巨大的梯度跳变。PPO/GRPO 在这种状态下会“漂移 25k 步学不到东西”。
4. **没有教师锚定**。SFT 有 a^* 把策略钉在专家分布上；RL 只有“最大化 reward”，没有标准答案，于是大幅更新可以把策略推向任意能骗到 reward 的方向（reward hacking），其副作用就是视觉/语言通用能力的**灾难性遗忘**。

灾难性遗忘的形式化代理指标是 **fine-tune 前后策略分布的 KL 散度**（“RL’s Razor”，Shenfeld et al. 2025）：更新引起的 $D_{\text{KL}}(\pi' \parallel \pi)$ 越大，通用能力损失越大。LLM/VLM 用 RLHF 的 KL penalty 显式压住这一项；但 VLA 既要保视觉又要保语言、KL 系数无从统一标定——这是一个尚未解决的 open problem。再叠加一个工程事实：直接 fine-tune 一个 VLA（OpenVLA-OFT, batch 8）需约 **62.5 GB 单卡显存**，消费级硬件根本跑不动。

正因如此，PLD 与 RECAP 都不**直接对 VLA 跑 PPO**，而是各自设计了一条“把 RL 变得可行”的迂回路线。理解这两条路线，要先理解它们共用的底层机制。

3 支撑机制

PLD 与 RECAP 看似差异巨大，却建立在同一组离线/离线-到-在线 RL 机制之上。本章把这些机制的目标函数与设计动机讲清楚，作为后两章的前提。

3.1 离线 RL 的核心病理：OOD 动作的 Q 值高估

这是理解 Cal-QL、RLPD、AWAC 的共同出发点。给定行为策略 π_β 采集的数据集 $\mathcal{D} = \{(s, a, r, s')\}$ ，标准 Bellman 备份目标是

$$\mathcal{B}^\pi \bar{Q}(s, a) = r(s, a) + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s')} [\bar{Q}(s', a')].$$

当策略改进步 $\pi(s) = \arg \max_a Q_\theta(s, a)$ 选出的 a' 落在数据分布之外（OOD）， $\bar{Q}(s', a')$ 是函数逼近器在**未见区域的外推值**，没有任何数据约束。 $\arg \max$ 会**系统地挑中那些被高估的 OOD 动作**，高估值经 Bellman 备份回传、进一步抬高，形成正反馈 $\rightarrow$ Q 值单调爆炸、训练发散。

为什么离线比在线更致命：在线 RL 中策略选了被高估的坏动作，环境会用真实低回报纠偏；离线设定无法采新数据，错误外推**永远得不到修正**。真机 VLA 恰恰处在“采数据极贵、近乎离线”的处境——这正是 PLD 必须用保守/稳定机制的根本原因。

应对这一病理有三条范式，分别被本报告的两个主角采用：**显式保守**（CQL $\rightarrow$ Cal-QL, PLD 用）、**隐式正则 + 架构抑制**（RLPD, PLD 用）、**隐式策略约束**（AWR/AWAC, RECAP 的理论近亲）。

3.2 CQL $\rightarrow$ Cal-QL：保守价值学习与“校准”

CQL 的保守目标在 TD 误差外加一个双向正则项：

$$\min_{\theta} \underbrace{\alpha \left(\mathbb{E}_{s \sim \mathcal{D}, a \sim \pi} [Q_\theta(s, a)] - \mathbb{E}_{s, a \sim \mathcal{D}} [Q_\theta(s, a)] \right)}_{\text{push down OOD; push up data}} + \frac{1}{2} \mathbb{E}_{\mathcal{D}} [(Q_\theta - \mathcal{B}^\pi \bar{Q})^2].$$

第一项压低当前策略选出的（可能 OOD 的）动作的 Q 、抬高数据集动作的 Q ，使 Q_θ 成为真值的下界，从而 $\arg \max$ 不会选到被高估的 OOD 动作。

Cal-QL 的“校准”改进针对 CQL 的一个副作用：CQL 常把 Q 压得过低（远低于真值）。一旦进入在线微调，智能体的探索动作即便真实回报很差，仍可能高于被严重低估的离线 Q 值，于是坏动作“看起来最优”，策略被误导去遗忘已学好的离线初始化——表现为 offline-to-online 早期的性能塌陷 (dip)。Cal-QL 的修复是把 push-down 项截断在一个参考价值 V^μ （如行为策略的 return-to-go，无 bootstrap 误差）之上：

$$\mathbb{E}_{s \sim \mathcal{D}, a \sim \pi} \left[\max(Q_\theta(s, a), V^\mu(s)) \right] - \mathbb{E}_{s, a \sim \mathcal{D}} [Q_\theta(s, a)].$$

即“ Q 只在高于参考值时才被压低”。这保证 $Q_\theta \gtrsim V^\mu$ （校准，防过度保守）同时仍是真值下界（保守，防高估）。机制上，在线探索遇到的坏动作（真实回报 $\leq V^\mu$ ）不再显得比离线策略更优 → 消除初始 dip、不浪费离线初始化、加速在线改进。这正是 PLD 选 Cal-QL 而非 CQL/IQL 的原因（PLD 的消融显示 Cal-QL > IQL > CQL，CQL 出现严重遗忘）。

3.3 RLPD：用四个最小设计在不加保守项的前提下稳住 off-policy 学习

RLPD 的立场与 Cal-QL 互补：不做离线预训练、不加显式保守项，仅靠四个设计选择就让 naive 的 SAC + replay 既稳又高效。这四件套被 PLD 的 Stage 1 完整继承。

- **对称采样 50/50**。每个 minibatch 一半来自离线 buffer、一半来自在线 buffer。每个梯度步恒有 50% 离线转移，Bellman 备份持续把 Q_θ “钉”在数据覆盖区域 → 相当于一个**始终在线的隐式正则**，无需写出显式约束。消融显示对比例不敏感，但 50/50 在稀疏奖励任务上方差最低（80/20 会因在线新鲜样本不足而退化）。
- **Critic 中的 LayerNorm**。对 critic 中间表示做 LayerNorm 后， $\|Q_{\theta, w}(s, a)\| \leq \|w\| \|\psi_\theta(s, a)\| \leq \|w\|$ ，即 Q 的输出被权重范数上界，即便动作远离数据支撑。这从根上**封顶了 OOD 外推爆炸**——这是 RLPD 不用显式保守项也能稳的关键。注意它只压住数值外推、不约束探索，所以既防发散又保留向未知高价值区域探索的能力。很多复现失败正是漏掉了这一行。
- **Ensemble + Min-Q**。维护多个 critic，每步随机取子集、用其中**最小值**做备份目标 $y = r + \gamma \min_{i \in \mathcal{Z}} Q_{\theta'_i}(s', \tilde{a}')$ 。取 min 相当于取悲观分位，系统性抵消“max 优化 + 函数逼近”带来的正向高估偏置。
- **高 UTD (update-to-data ratio)**。每与环境交互一步执行 G 次梯度更新（ G 可达 10–20）。每条新数据被反复 bootstrap，价值信息回传更快 → 样本效率大增。但高 UTD 等于“对同一小批数据做大量梯度步”，统计上极易过拟合、放大高估正反馈——所以它**必须依赖前三者才不散**。四要素是一个协同整体：UTD 提供效率，前三者提供让该效率可用的稳定性。

值得强调的接口：Cal-QL 在追求高 UTD 时直接复用 RLPD 的 critic 架构 (LayerNorm + ensemble)。PLD 正是把“Cal-QL 的保守校准”与“RLPD 的架构稳定 + 对称采样”缝在一起，这才有了第 3 章的 Stage 1。

3.4 AWR / AWAC：advantage-weighted regression (RECAP 的理论近亲)

AWAC 从一个带 KL 信赖域的策略改进问题出发： $\max_{\pi} \mathbb{E}_{a \sim \pi} [A^{\pi_k}(s, a)]$ s.t. $D_{\text{KL}}(\pi \| \pi_\beta) \leq \epsilon$ 。其 KKT 解 $\pi^* \propto \pi_\beta \exp(A^{\pi_k}/\lambda)$ 投影回参数策略类，得到可实现的 **advantage-weighted regression**：

$$\theta_{k+1} = \arg \max_{\theta} \mathbb{E}_{(s, a) \sim \mathcal{D}} \left[\log \pi_\theta(a|s) \cdot \exp\left(\frac{1}{\lambda} A^{\pi_k}(s, a)\right) \right].$$

这是一个用 $\exp(A/\lambda)$ 给数据重新加权的**监督学习**。它的三个性质正是 RECAP 哲学的来源：(i) 解析解以 π_β 为基底、回归只在数据样本上做，结果策略**天然被拉向数据支撑**——这是 KL 约束的隐式实

现，从不查询 OOD 动作的 Q，回避了 2.1 的病理；(ii) advantage 用 off-policy 的 bootstrap Q 估计，可复用任意数据；(iii) 权重直接作用在采样的 (s, a) 上，无需重要性采样、无需显式拟合 π_β 。

RECAP 与 AWR 共享内核(用 advantage 引导策略偏向高优势动作)，但实现不同：AWR 把 advantage 压成回归权重 $\exp(A/\lambda)$ (一种软筛选，会下采大量低权重数据)；RECAP 把 advantage 作为生成模型的条件输入，并在推理时用 CFG 调节——这把“优势 → 动作”建模为一个可在推理期连续调控的条件生成分布，而非训练期固化温度的加权回归。

3.5 Classifier-Free Guidance (CFG)

CFG 来自扩散/生成模型文献，是 RECAP advantage-conditioning 的推理机制。要从条件分布 $p(x|c)$ 采样 (RECAP 中 x =动作、 c =目标 advantage)，CFG 用同一网络联合训练条件模型 $p_\theta(x|c)$ 与无条件模型 $p_\theta(x)$ (训练时以一定概率随机丢弃条件 c)，推理时按

$$\hat{p}(x|c) \propto p(x) \cdot \left(\frac{p(x|c)}{p(x)} \right)^\beta$$

外推组合，等价于在 score 空间做线性外推 $\tilde{\epsilon} = \epsilon_\theta(x, \emptyset) + \beta(\epsilon_\theta(x, c) - \epsilon_\theta(x, \emptyset))$ 。 $\beta > 1$ 时放大“条件方向”，把采样推向更强满足 c 的样本，同时以无条件先验 $p(x)$ 为底盘保证样本仍落在数据流形上。当 c =高 advantage 时，这就是一个推理时可连续调节的策略改进旋钮： β 越大越偏向高优势动作， $\beta \rightarrow 0$ 退回行为先验 (更安全)。它与 LayerNorm/隐式 KL 在精神上的一致——在不显式查询 OOD 的前提下偏向高价值动作。

4 PLD：用残差 RL 做“部署对齐的数据生成”

论文：Self-Improving VLA Models with Data Generation via Residual RL (Xiao et al., NVIDIA/CMU/UC Berkeley/UT Austin, arXiv:2511.00091, ICLR 2026 Poster)。

PLD 的真正新颖点不是残差策略、也不是双 buffer (这些分别来自 ResiP 与 RLDP)，而是用 off-policy 残差 RL 做“部署对齐的数据策展 (data curation)”，再把数据蒸馏回 VLA，训完即弃专家。下面逐层拆解。

4.1 残差策略的精确形式，以及为什么是残差而非 LoRA

执行动作是 VLA 基础动作与一个受限残差之和：

$$\bar{a} = a_b + a_\delta, \quad a_b \sim \pi_b(\cdot | s), \quad a_\delta \in [-\xi, \xi]$$

其中 π_b 是冻结的 VLA， a_δ 来自一个任务专属的高斯残差策略 $\pi_\delta(\cdot | s, a_b)$ ——它“看到 VLA 想做什么 (a_b) 再决定如何修正”。残差网络是个 3 层 MLP (hidden 256、Tanh、LayerNorm)，视觉用冻结的小 ResNet 编码，单 task 训练峰值仅约 5 GB 显存，RTX 4090 可跑 (残差本体约 10M 参数，为解读量级)。

残差幅度被 clip 到 $[-\xi, \xi]$ 是一个双刃设计： ξ 太大，早期更新让组合策略大幅偏离 π_b 、诱发不稳定探索、早期性能崩； ξ 太小，探索不足、渐近上不去。消融取 LIBERO $\xi = 0.5$ 、SimplerEnv $\xi = 0.1$ ，把残差锁定为“轻微修正”而非“取代 VLA”。

为什么用残差而不是 LoRA 或直接 fine-tune，这是 off-policy 复用的关键：残差是一个结构独立的高斯策略，可以用任何现成 off-policy 算法 (SAC/Cal-QL) 训练 → 可维护 replay buffer、老 rollout 反复用。而 LoRA 嵌在 VLA 内部，反向传播仍要穿过整个 VLA (显存照炸)，且直接对 flow/diffusion 动作头做“最大化 Q”的 RL 极其困难。更重要的是，冻结主干从结构上根除了灾难性遗忘——残差即便被训坏也无所谓，它本就是用完即弃的工具。这一点是 PLD 整个设计的支点。

4.2 Stage 1 (Probe + Learn): Cal-QL critic + 残差 actor

critic 用 Cal-QL (见 2.2) 学习, 目标即

$$\min_{\theta} \alpha \mathbb{E}_{s \sim \mathcal{D}, a \sim \bar{\pi}} [\max(Q_{\theta}(s, a), V^{\mu}(s))] - \frac{1}{2} \mathbb{E}_{s, a \sim \mathcal{D}} [(Q_{\theta}(s, a) - \mathcal{B}^{\bar{\pi}} \bar{Q}(s, a))^2],$$

bootstrap 用**组合策略** $\bar{\pi}$ 。残差 actor 通过最大化 SAC target 更新 (带 entropy 正则), **刻意不加 BC 正则**——这样训出的专家 $\bar{\pi}$ 不被数据质量或 base 性能拖住, 能真正超越 60% 的起点。critic 与 actor 的更新比为 2:1 (温和 UTD, 让价值先于策略收敛), 并用 RLPD 的 LayerNorm + 双 Q + min。

一个易被忽略但关键的设计是 **warmup critic**: 先只用 π_b 的数据初始化 Cal-QL critic、actor 不动。原因有二——直接同时训 Q 和残差会“两个都在乱猜”训不起来; 且消融显示 Cal-QL 预训练 critic 渐近性能更优、对保守系数 α 鲁棒。

Stage 1 全程**冻结 VLA**: 组合密度 $\bar{\pi}(\cdot|s) = \pi_b(\cdot|s)\pi_{\delta}(\cdot|s, a_b)$, 梯度只回传到 π_{δ} 与 Q_{θ} , VLA 只做 forward 提供 a_b 。

双 buffer 与样本效率: 维护离线 buffer (π_b 的成功 rollout, 永久锚定“成功长什么样”) 与在线 buffer (组合体跑出的全部 transition), 每个 minibatch 各取一半 (对称采样)。离线一半保证 Q 不会因在线早期全是失败样本而把所有动作判烂; 在线一半提供新鲜探索信号。配合 off-policy 复用——buffer 容量 250k、训练约 250k actor steps——**一条真机 rollout 进 buffer 后会参与约 250k 次梯度更新**, 这就是“把每条 rollout 榨干两到三个数量级”的来源。真机实测: Franka 上 200 条 teleop SFT 后, 2 小时内 PLD-RL 达到 100% 成功率。

4.3 Stage 2 (Probing): PLD 真正的灵魂

Stage 1 只是把残差专家训出来; Stage 2 才是 PLD 区别于一切“采成功数据再 SFT”方法的地方。混合 rollout 的控制律是:

$$\pi(s_t) = \begin{cases} a_{\text{base}}, & t < \alpha T \quad (\text{只有冻结的 VLA 控制, “探针”}) \\ a_{\text{base}} + a_{\delta}, & t \geq \alpha T \quad (\text{残差接管, “救场”}) \end{cases}$$

前 αT 步只让 VLA 自己走——它会自然漂进自己常翻车的状态; 后 $(1 - \alpha)T$ 步残差接管把困境救回成功。收集到的轨迹因此是“起点采样自 VLA 自己的访问分布 + 后段是专家从次优区恢复的纠错序列”。注意探针步只用于状态初始化、不进 replay buffer (它制造起点, 但不污染 critic 学的高价值样本)。

为什么这样能防遗忘——用 support / 分布覆盖的语言: 若全程用组合体 $\bar{\pi}$ 采 (即 $\alpha = 0$), 数据是“高度最优、无犹豫、单峰”的专家轨迹, 集中在状态空间的最优窄带; 而 VLA 部署时真正会访问的 OOD / 失败状态在这种数据里 **underrepresented**。用这种数据 SFT, VLA 学到的是“如何在最优带里走”, 却没解决任何它实际会遇到的失败。Probing 把数据的 support 拉回 VLA 自己的访问分布, 使后续 SFT 只需在 VLA 已经熟悉的状态上做小幅修正。

α 的消融给出了精确的 trade-off: 随 α 增大, 成功轨迹需要更长的“绕路”来纠正 base 的次优行为, fine-tuning 性能在 $\alpha = 0.6$ 处 **plateau**, 超过 0.6 后下降 (探得太深, 残差也救不回)。在数据来源对比上 (LIBERO-90, π_0 SFT, 按 coverage ratio 的总成功率):

coverage	PLD Data (带 probing)	Base Rollout ($\alpha=0$)	Human Data
0.1	0.314	0.103	0.272
0.3	0.470	0.068	0.419
0.6	0.637	0.328	0.611
1.0	0.871	0.488	0.815

PLD 数据全面优于“纯 base rollout”和“人类示教”，且在未见任务上 10% coverage 即有 24.4% zero-shot 成功率，而纯 base rollout 几乎不泛化。

4.4 Stage 3 (Distill): 普通 SFT, 但防遗忘机制在 Stage 2

Stage 3 没有任何新算法, 就是把 Stage 2 数据 $\mathcal{D}^{\text{PLD}}$ 喂进 VLA 的标准 SFT (按动作头实例化为 token NLL 或 flow-matching MSE):

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(s, a^*) \sim \mathcal{D}^{\text{PLD}}} [\log \pi_{\text{VLA}}(a^* | s; \theta)].$$

核心论点——为什么“防遗忘机制在 Stage 2 而非 Stage 3”, 可以从 SFT 梯度幅度讲清楚。灾难性遗忘的根因不是“SFT 这个算法”, 而是 **SFT 数据离当前策略分布太远** → **BC loss 初值大** → **梯度大** → **参数大幅更新** → **共享参数上的通用能力被覆盖**。PLD 数据由 probing 生成、**起点就在 π_b 自己的访问分布上**, 于是对每个 s , 目标 a^* 与 $\pi_b(\cdot|s)$ 的差异很小 (仅救场段有适度偏移), $-\log \pi_{\text{VLA}}(a^*|s)$ 初值就小、梯度小、更新温和, 通用能力得以保留。用 KL 语言: PLD 数据 **biased toward base policy** → SFT 诱导的 $D_{\text{KL}}(\pi' \parallel \pi_b)$ 小 → 遗忘少。

这与“在 SFT 阶段冻结关键参数”(如 MIFO 的事后补救) 路线**本质不同**: PLD 让 SFT 用的数据本身就不需要大幅更新, **从数据侧根除遗忘**。蒸馏把“残差专家 + 救场能力”永久写进 VLA 权重, 部署时 π'_{VLA} 在原本会失败的状态直接输出纠错动作, **不再需要残差、Q 函数或 RL, 零额外推理开销**。

4.5 实验关键数字与技术局限

性能: LIBERO 上 π_0 93.4→97.2 (Goal 子集 +7.9)、OpenVLA 91.8→99.2; SimplerEnv 平均 71.8→96.6 (Pick Carrot +50.6); 真机 peg insertion 30/30、cluttered 未见物体抓取 PLD 28/30 vs 人类数据 12/30。失败模式分析很说明问题: 用人类/RLPD 数据训出的策略常把方块推进角落卡死 (因为这两类数据**从没访问过角落状态**), 而 PLD 显式探针到这些状态并生成 recovery, 故能救回。

真实的技术局限 (剥离 PPT 话术后):

1. **只用稀疏二元奖励**。Cal-QL critic 学 $\{0, 1\}$ 信号, 在 reward classifier 难训的任务上缺乏梯度密度。
2. **base 必须够强 (competence threshold)**。残差只能在 a_b 邻域 $[-\xi, \xi]$ 内修正, 且 probing 依赖“base 能走到有意义的状态”。消融: base >80% 可冲到 99%, base <50% 救不回。
3. **closed-loop 残差 vs open-loop chunking 蒸馏的冲突**。训练时残差是逐步闭环控制, 蒸馏后却用 action chunking 开环执行——闭环的反应式纠错被“打包进 chunk”硬塞给开环学生, 高动态接触任务会失效。
4. **per-task 残差, 跨任务无 skill 共享**。每个 task 训独立残差 MLP, 90 个 task = 90 个专家, 训练侧成本随任务数线性增长。
5. **flat replay buffer, 无能力感知**。在线 buffer 是简单 FIFO, 不按 TD-error/语义价值优先保留, 长期部署会累积冗余。
6. **episode-level 信用分配**。reward 只在末尾给 0/1, step 级贡献全靠 Q 隐式 bootstrap, 放大了对双 buffer 与 LayerNorm 的依赖。

这些局限中的 (3)(4)(5)(6) 不是实现瑕疵, 而是**范式层面的技术空白**, 第 6 章会看到 RECAP 也有同构的空白。

5 RECAP: 把 advantage 从 loss 通道挪到 conditioning 通道

论文: Physical Intelligence, $\pi^*0.6$: a VLA That Learns From Experience, arXiv:2511.14759 (未开源)。RECAP = RL with Experience and Corrections via Advantage-conditioned Policies。

RECAP 是一个**迭代式 offline RL** 配方（论文反复强调 “iterated offline RL” 而非 online RL）。它的全部巧思可压缩成一句：**把 RL 的 advantage 信号从 “loss 通道” 挪到 “conditioning 通道”**。

5.1 三阶段，以及为什么每轮从 anchor 重训

算法是 PRETRAIN $\rightarrow$ SPECIALIZE $\rightarrow$ ITERATE 三阶段：PRETRAIN 一次性在海量多机器人数据上训出 value V_{pre} 与策略 π_{pre} （即 $\pi_{0.6}$ ）；SPECIALIZE 为新任务从 $V_{\text{pre}}/\pi_{\text{pre}}$ 微调出起点；ITERATE 循环 K 轮（用当前策略采数据并入池 $\rightarrow$ 重训 $V \rightarrow$ 重训 π ）。

最反直觉的设计是：ITERATE 每一轮的 V 和 π 都明确 “**from V_{pre} / from π_{pre}** ” ——从预训练 **checkpoint** 重新微调，而不是从上一轮增量更新。论文给出的理由是 “避免多轮迭代的 drift”。机制层解释：若从 π^{k-1} 增量更新，每轮都在一个已偏向单任务的策略上再做单任务微调，分布偏移逐轮累积——这正是连续微调下灾难性遗忘的具体形态。把 $\pi_{\text{pre}}/V_{\text{pre}}$ 当**永久 anchor**、每轮 “回到原点再出发”，则唯一在变的是**数据池 $\mathcal{D}_\ell$** 越来越大越来越好，改进完全由**数据质量**驱动而非由**参数漂移**驱动 $\rightarrow$ 多轮迭代单调可控。代价是每轮要付完整微调成本，但 PI 有集群可接受。

这与 PLD 形成有趣对照：PLD 靠 “冻结主干” 在结构上防遗忘，RECAP 靠 “每轮回到 anchor” 在流程上防遗忘——两种不同的防遗忘哲学。

5.2 从 0/1 标签到 step reward，到分布式 value

每条 episode 只需人标 1 bit（成功/失败），由此导出 step reward：

$$r_t = \begin{cases} 0 & t = T \text{ 且 success} \\ -C_{\text{fail}} & t = T \text{ 且 failure} \\ -1 & \text{otherwise} \end{cases}$$

其中 C_{fail} 是一个大常数（精确值论文未给）。在无折扣下，一条**成功轨迹**的 Monte-Carlo return $R_t(\tau) = \sum_{t' \geq t} r_{t'}$ 恰好等于 $-(\text{到成功的剩余步数})$ 。换言之，**value function 学的物理量就是 “到成功的负剩余步数”**，归一化到 $[-1, 0]$ （ $V \approx 0$ 已近成功， $V \approx -1$ 最坏）。失败轨迹未步多了 $-C_{\text{fail}}$ ，其 return 被显著拉低，于是 value 被迫**显式学会识别失败状态**——这正是 RECAP 保留失败 episode 的意义。

value function 用一个小一号的 VLM backbone（约 670M）实现，但**输出不是标量，而是 V 在 201 个离散 bin 上的 softmax 分布**，训练目标是对离散化 MC return 做交叉熵：

$$\min_{\phi} \mathbb{E}_{\tau \in \mathcal{D}_\ell} \left[\sum_{o_t \in \tau} H(R_t^B(\tau), p_{\phi}(V | o_t, \ell)) \right], \quad V(o_t) = \sum_b p_{\phi}(V = b | o_t) v(b).$$

为什么用分布式回归 + 交叉熵而非 MSE 标量回归：(i) 真实 return 分布常多峰（同一状态有时离成功近、有时会失败），MSE 只能压成条件均值、丢掉模态信息，201-bin softmax 保留整个分布形状；(ii) 交叉熵把回归变成分类，目标是固定 one-hot、梯度天然有界、对失败轨迹的极端负 return 鲁棒，而 MSE 对长尾大残差会产生大梯度、训练易震荡——在大 VLM backbone 上训 critic 时这种稳定性差别是决定性的。

为什么用 Monte-Carlo 而非 TD/Q：TD bootstrap 在 “大模型 + 异质 off-policy 数据 + 稀疏奖励” 下极易发散（需要 target network、ensemble、保守项一整套，正是 PLD 用 Cal-QL 的原因）。MC return 无 bootstrap、无自举发散风险，代价是方差较高、且 value 反映的是**行为策略的期望 return**（而非最优 V^* ）。但因为 value 只被用来**算 advantage 做条件**（不进 actor 的反向传播），这点偏差可容忍——RECAP 不需要精确的最优 critic，只需要一个**可靠的、能区分 “好动作 vs 坏动作” 的相对信号**。这是它与 PLD 在 credit assignment 上的根本分歧（PLD 要一个精确 Q 来 arg max，所以必须 Cal-QL；RECAP 只要一个相对排序，所以 MC 足矣）。

5.3 advantage、二值 indicator 与三类数据的不对称处理

advantage 的精确形式比 “ $R_t - V$ ” 更细：后训练阶段用 N 步 lookahead (论文 $N = 50$):

$$A^\pi(o_t, a_t) = \sum_{t'=t}^{t+N-1} r_{t'} + V^\pi(o_{t+N}) - V^\pi(o_t),$$

预训练阶段才退化为纯 MC ($N = T$, 只需一次 value 推理、便于在海量数据上摊销)。直观含义是“这一步实际带来的进展, 减去 value 预期的进展”。

策略不直接吃连续 advantage, 而是吃一个二值 **optimality indicator** $I_t = \mathbb{1}[A^{\pi_{\text{ref}}}(o_t, a_t, \ell) > \epsilon_\ell]$, 阈值 ϵ_ℓ **per-task 自动校准** (预训练设成约 30% 样本为正、微调约 40%、demo 质量极高的折衣任务调到约 10%)。为什么用阈值而非连续 advantage 或推理期 β : 原版 CFGRL 用推理期 guidance 锐化, 但高 guidance 会“把动作推到支撑集边缘产生危险动作”, 所以 RECAP 把“该多优化”这个旋钮从推理期挪到训练期的 ϵ_ℓ , 更可控。

三类数据段填 I_t 的方式刻意不对称:

数据段	I_t	理由
demo (人工示教)	强制 True	视为理想动作, 绕过 value 评估
human correction (自主中专家接管)	强制 True	假设专家纠正总是好的
autonomous (成功/失败均保留)	$\mathbb{1}[A > \epsilon_\ell]$	真按 value 审判; 失败段多为 False, 教策略“别这么做”

人类来源的动作被无条件信任、绕过 value; 只有机器自主动作才接受 value 审判。失败 episode 不被丢弃——它既训 value (提供失败负样本), 又以 $I_t=\text{False}$ 的身份教策略避错。这是 RECAP 比“只用成功数据”信息利用率更高的根源。

5.4 策略训练: CFG-style 双支目标 + prefix 注入 + random drop

策略目标是一个条件/无条件双支 NLL:

$$\min_{\theta} \mathbb{E}_{\mathcal{D}} \left[\underbrace{-\log \pi_{\theta}(a_t | o_t, \ell)}_{\text{unconditional}} - \alpha \underbrace{\log \pi_{\theta}(a_t | I_t, o_t, \ell)}_{\text{conditional}} \right].$$

第一支学“行为策略的平均” $\pi_{\text{ref}}(a|o, \ell)$, 第二支学“已知好坏标签条件下”的策略 $\pi_{\text{ref}}(a|I, o, \ell)$ 。本质是对全部数据(成功+失败、好+坏动作都用)做的加权/条件监督学习——没有 policy gradient、没有 importance ratio、没有 KL penalty。坏动作不是被删, 而是被打上 $I_t=\text{False}$ 照样训。

advantage 怎么注入而不动架构: 把 indicator 实现为一段文本 prefix—— $I_t=\text{True}$ 时输入 “Advantage: positive”、否则 “Advantage: negative”, 位置放在任务语言 ℓ 之后、动作 token 之前, 于是只影响动作的 log-likelihood。因为 VLA 本来就是 VLM、输入序列里已有语言, “多两个 token” 零成本, 且预训练时模型已见过这段语义、新任务上零成本迁移。

random drop I_t (论文为 30%, 注: 精读笔记误记为 50%): 训练时随机丢弃 advantage 条件——30% 步在训 unconditional 支、70% 步在训 conditional 支。这一个 trick 同时训出两个模型、省掉一遍 forward, 正是 CFG 在训练侧的标准做法; 它等价地替代了目标里的权重 α (不用显式算两项加权, 靠随机 mask 把两支信号混进一个 minibatch)。

5.5 CFG 推理与 flow-matching 下界

从正则化 RL 出发, advantage-conditioned 改进策略有闭式

$$\hat{\pi}(a | o, \ell) \propto \pi_{\text{ref}}(a | o, \ell) \left(\frac{\pi_{\text{ref}}(a | I, o, \ell)}{\pi_{\text{ref}}(a | o, \ell)} \right)^\beta.$$

$\beta = 1$ 时退化为“直接用 conditional 支”——**单次 forward、零额外开销** (默认部署); $\beta > 1$ 时把“improvement 方向”额外锐化, 需要 conditional/unconditional 两次 forward。因为 $\pi_{0.6}$ 是 flow-matching 策略、学的是 score, 推理时按 $\nabla_a \log \pi_\theta(a|o, \ell) + \beta(\nabla_a \log \pi_\theta(a|I, o, \ell) - \nabla_a \log \pi_\theta(a|o, \ell))$ 采样即可——**与图像扩散的 classifier-free guidance 形式完全一致**, 只是把“类别条件”换成“Advantage: positive”。

由于 flow-matching 没有解析 $\log \pi$, 论文用扩散 $\leftrightarrow$ flow 等价把 flow-matching MSE 当作 log-likelihood 的下界来优化 (离散 token 的自回归似然 + 连续动作的 flow-matching 加权 MSE 共同构成下界)。这解释了为什么 RECAP 这套“似然式”目标能套在没有解析似然的 flow VLA 上。

5.6 为什么绕开了 PPO: AWR 视角与单调改进下界

RECAP 本质属于 AWR/AWAC 一族 (见 2.4), 但它不用那个广为人知的 $\exp(A/\beta)$ 加权解 (论文指出 AWR 会“丢弃或大幅下采相当一部分数据, 沦为 filtered imitation learning”), 而是用一个**少为人知的结果**: 定义改进策略 $\tilde{\pi}(a|o) \propto \pi_{\text{ref}}(a|o) p(I|A)$, 只要 $p(I|A)$ 是 advantage 的**单调递增函数**, 就保证 $\mathcal{J}(\tilde{\pi}) \geq \mathcal{J}(\pi_{\text{ref}})$ ——**改进策略不差于参考策略** (这是 RECAP 单调改进的理论依据)。对 I 取阈值 delta 分布并用 Bayes 重写, 就得到上面的闭式与双支 NLL。整个推导只涉及监督式对数似然, **advantage 从不进梯度**。与 PPO 的本质区别:

维度	PPO (on-policy PG)	RECAP (advantage-conditioned SL)
数据	on-policy, 几步就要采新数据	off-policy/全离线, demo+ 历轮 + correction 全用
advantage 进哪	进 loss/梯度 $\nabla \log \pi \cdot A$	进 conditioning 通道 (prefix 文本)
importance ratio	需 $\pi_\theta/\pi_{\text{old}} + \text{clip}$	无
KL / 信任域	需 KL penalty 或 clip	无显式 KL (隐含在“从 anchor 重训 + 阈值”里)
flow VLA 兼容	PG 需解析 $\log \pi$, flow 没有 $\rightarrow$ 难	用下界天然兼容

实验直接对比显示 PPO (为稳住 off-policy 不得不把信任域收到极小) 与 AWR 都显著差于 RECAP。

5.7 迭代改进的微观机制: I_t 会翻转

这是 RECAP “持续改进”的形式化引擎。数据池单调增长、旧 episode 的 (o_t, a_t, r_t) 永不改变, 但每轮都用更大的 $\mathcal{D}_\ell$ 从 V_{pre} 重训出新的 V^k , 于是

$$A^{V^k}(o_t, a_t) \neq A^{V^{k-1}}(o_t, a_t) \implies I_t^{(k)} = \mathbb{1}[A^{V^k} > \epsilon_\ell] \text{ 可能 } \neq I_t^{(k-1)}.$$

同一 step 的 optimality 标签在不同轮可能翻转。随策略变强, 行为策略的平均水平 (即 V) 抬升, 一个上一轮还算“高于平均”的动作, 这一轮可能已“低于新的更高平均”。这构成一个**自举式的标准提升**: 策略不断把自己的水平当作新的及格线, advantage 永远衡量“相对当前水平的进步”, 加上阈值 ϵ_ℓ 按“约 40% rollout 为正”动态校准, 及格线随进步自动上移——这就是 RECAP 跨多轮持续涨而不饱和的微观引擎。

5.8 实验关键数字

真机静态双臂、50 Hz, 三大任务族 (咖啡/装箱/折衣)。标志性能力: **制作意式咖啡连续运行 13 小时** 不中断、折陌生衣物 2 小时 +、工厂装纸箱 600 秒全流程。核心改善: 最难任务上 throughput (成

功/小时) 相对 $\pi 0.6$ baseline **翻倍以上**、failure rate **约减半**，多数任务最终落在 90%+。数据量给出了人力刻度：T 恤折叠每轮 4 机器人采 300 条、2 轮、**0% intervention**；而长程装箱每轮 600 demo + 360 correction，是 intervention 比例最高的任务。关键 ablation：advantage conditioning 这一项相对“无 indicator 的 SFT”有清晰增量；同数据下 RECAP 远超 AWR 与 PPO；默认 $\beta = 1$ 已达报告性能。

6 PLD 与 RECAP：机制级对比

两者都在回答同一个问题——“真机上 rollout 极贵、直接 RL 不可行，如何自改进？”——但给出的是两种正交的答案。

1. 数据生成 vs 信号生成，是两种“把 RL 变可行”的哲学。PLD 让 RL 的产物是一批分布对齐的**数据**，RL (残差 Cal-QL) 只在幕后短暂存在、训完即弃，最终交付的是经过普通 SFT 的单一 VLA。RECAP 让 RL 的产物是一组 **step 级 advantage 信号**，把它编码成 conditioning 注入策略，用监督学习消费。前者把复杂性集中在“如何采到对的数据”，后者集中在“如何把弱标签升级成可用信号”。

2. credit assignment：精确 Q vs 相对 value。PLD 需要一个**精确的 Q** 来 $\arg \max$ 训残差，所以必须背上 Cal-QL 的保守校准 + RLPD 的全套稳定 trick (LayerNorm/min-Q/对称采样/高 UTD) 来对抗 OOD 高估。RECAP 只需要一个**相对的、能排序好坏的 value**，所以敢用最简单的 MC + 分布式回归，彻底躲开 TD 发散。这是二者全部工程差异的源头。

3. 防遗忘：数据侧 vs 流程侧。PLD 在**数据侧**防遗忘——probing 让 SFT 数据贴着 base 分布、诱导的 KL 小、梯度小。RECAP 在**流程侧**防遗忘——每轮从 π_{pre} anchor 重训、不让参数逐轮漂移。两者都不靠显式 KL penalty，却殊途同归地把“更新引起的分布偏移”压住。

4. 样本效率 vs 算力。PLD 是**样本效率优先**：off-policy 双 buffer 把每条真机 rollout 复用约 250k 次，5 GB 显存消费级可跑，适合算力受限的学术/小团队。RECAP 是**算力换简单**：单池只增不减、每轮从头重训、brute-force 收数千 episode，依赖 PI 的集群，工业直接适用、长程连续运行能力更强。

5. 推理与部署。PLD 蒸馏后零额外开销 (部署就是原 VLA)。RECAP 部署默认 $\beta = 1$ 单次 forward，仅多两个 prefix token；要更强可开 CFG 双 forward。

共同的技术空白 (注意是真实的技术根因，不是“覆盖了几条框架”)：

- **失败 rollout 的 step/段级利用不充分**。PLD 直接丢失败 episode；RECAP 虽保留失败 episode，但 advantage 由 value 在 episode 内逐步算——对一条“前 80% 子任务都对、最后装歪导致整体失败”的长 episode，失败末端会把整段的 value 拉低，“局部正确”的段难以被干净地识别为正样本。
- **跨任务 skill 无显式复用**。PLD per-task 残差，RECAP per-task specialize，都没有把“磨豆”“压粉”这类可跨任务复用的 skill primitive 显式抽出来。
- **buffer 缺乏能力感知的主动管理**。PLD 在线 buffer 是 FIFO，RECAP 单池只增不减——都没有“按能力评估主动归档/遗忘过时失败、对复发情形重新激活”的机制，长期持续部署会累积冗余。
- **长 horizon 的闭环-开环鸿沟**。PLD 把闭环残差蒸馏进开环 chunk；RECAP 靠 episode 级信号 + 多轮迭代缓解，但都未真正解决长程时序信用分配。

7 由开放问题导出的研究方向：HEARS-Buffer

上一章的四个共同空白，本质都指向同一个被忽视的层次：**经验数据的存储与利用层**。PLD 和 RECAP 各自创新了“如何采” (probing) 与“如何转化信号” (advantage conditioning)，但都把存储退化成了最朴素的形式 (FIFO / 单池)。HEARS-Buffer 的定位由此自然导出：它不是又一个

self-improving 算法或 VLA 模型，而是 self-improving 训练范式缺失的“第四个组件——经验数据层基础设施”，可同时挂载到 PLD 或 RECAP 之上（互补而非竞争）。

下面把四个开放问题映射成四个技术组件，其中**段级 SFT+RL 混训**直接针对最尖锐的“失败 rollout 利用”空白，是研究的差异化核心。

核心组件·段级 SFT+RL 混训。针对“局部正确但整体失败”的长 episode，先做**段切分**，再做**段级混训**。段切分用一个融合 cut score 在四个信号上找边界：

$$\text{cut_score}(t) = w_1|A(t)-A(t-1)|+w_2|V(t)-V(t-1)|+w_3\mathbb{1}[\hat{\ell}(t) \neq \hat{\ell}(t-1)]+w_4\mathbb{1}[\text{intervention}],$$

其中子任务预测 $\hat{\ell}$ 的变化是零成本的天然语义边界、人类接管/松手是 ground-truth 级边界（故 w_4 最高）。切出的段按段内平均 advantage 标三类质量：**good** ($\bar{A} > \epsilon$ ，纯 RL 强化 + 探索)、**bad** ($\bar{A} < 0$ 且 episode 失败，纯 SFT 朝一个 recovery target)、**unclear** (弱双修)。关键的一条特例规则正是对 RECAP 空白的补丁——**autonomous + 失败 episode + 段内 $\bar{A} > 0 \rightarrow$ 判为 good**：段级 advantage 能识别“被全局失败拖累的局部正确段”，而 RECAP 的 episode 级处理做不到。bad 段的 recovery target 由在 good 段库做 KNN（视觉 embedding 近邻，同 cluster/同 task 优先）检索得到，训练时朝它做 BC。最终落到 flow-matching VLA 的统一损失：

$$\mathcal{L}_\sigma = \text{SFT}_w \cdot \mathcal{L}_{\text{BC}}(\sigma, \text{recovery_target}) + \text{RL}_w \cdot \mathcal{L}_{\text{flow-adv}}(\sigma, I_{\text{adv}}),$$

其中 $\mathcal{L}_{\text{flow-adv}}$ 直接复用 RECAP 的 advantage-prefix conditioning。这相当于把 **RECAP 的推理级 advantage 信号下沉到段级训练，并补上 PLD 完全丢弃的失败数据利用**——是两条主线机制的一次嫁接。

其余三个组件（针对另外三个空白，多数可基于成熟开源库实现，非研究新点）：跨任务 skill 库（在 VLA 视觉 embedding 上做增量聚类 + 倒排索引，解决 skill 复用）；能力感知的生命周期管理（周期性 capability check + 主动归档/遗忘 + 对复发情形按“距归档质心近”自动重新激活，解决 buffer 主动管理）；运行时检索（FAISS 近邻，解决“上次在这个厨房成功的经验”这类检索增强决策）。底层存储用 LeRobot v3.0（Parquet+MP4，支持 append/压缩），三层压缩把单任务长期数据从 ~100 GB 压到 6–10 GB（端测可行）。

为什么必须先复现 PLD + RECAP 再做 HEARS：HEARS 是数据层基础设施、本身不产生 rollout，必须有 baseline 算法的真机 rollout 才能验证价值（提供对照、提供数据集、提供训练接口的参考实现）。建议的验证路径是 6 个月四阶段：复现 PLD baseline（4 周， 1×4090 ，低风险） $\rightarrow$ 复现 RECAP 的 value+advantage 部分（8 周，集群，高风险，因未开源，备选只复现 value function 用作段切分判据） $\rightarrow$ 段级混训 PoC（4 周，验收“段切分 F1>0.7、质量标注准确率 >75%”） $\rightarrow$ 完整集成 + 论文（8 周，验收“HEARS+PLD 相对 PLD baseline 成功率提升 $\geq 3\%$ ”）。目标会议 CoRL 2026 / ICLR 2027。最大的两个风险是 **RECAP 复现工程量**（未开源）与 **GPU 资源**。

8 训练动力学：第 1 章诸多论断的理论支撑

本章把第 1 章“SFT 与 RL 在真机上为何必须配合、为何 RL 能修复 SFT 遗忘”的论断，落到分布/梯度/熵三个机制层面。完整文献版见仓库 docs/01_训练动力学机制解读.md，此处给出与 VLA 直接相关的精炼。

分布层面：SFT 扩张、RL 锐化。SFT 最大化专家似然，把概率质量铺到老师走过的所有路径上（off-policy、扩张/同质化）；RL 最大化期望奖励，把质量从错误路径抽走、堆到对的路径上（on-policy、压缩/锐化）。二者方向相反，这是它们互补又互相覆盖的根因。但“SFT 记忆、RL 泛化”只是现象级总结——更准确的机制是：**SFT 的问题不在“记忆”而在“OOD 早峰后遗忘”，RL 很大程度是在“修复”而非凭空泛化**。谱分析显示，SFT 引起的遗忘对应权重矩阵奇异向量的旋转（方向漂移），而奇异值基本不变（幅度没丢）；RL 微调相当于把奇异向量旋回泛化方向的隐式正则。一个重要限制：

RL 只能从有限范围的 SFT checkpoint 修复——SFT 练过头就救不回来。这对 VLA 后训练的直接含义是：SFT 不要练到饱和，在 OOD 成功率达峰附近就切入 RL。

梯度层面：SFT 只有正梯度，RL 有负梯度。 SFT 的标准梯度等价于一个“隐式奖励 $1/p(y|x)$ ”的策略梯度——越不确定的 token 被赋越大权重（逆概率加权、无界方差），这是 SFT 泛化差的根因，可用一行 reward rectification 校正。RL 的梯度是 advantage 加权的：正优势推高、负优势压低，**负梯度产生的 squeezing effect 才是锐化分布的真正动力**，而 SFT 没有负梯度（只会抬对的、不会压错的）。统一视角下，SFT 不过是“优势恒为 +1、无负样本”的退化 RL——这解释了为什么单纯叠加 SFT 与 RL 会互相覆盖（MIFO 的观察：SFT 更新冗余幅度大，会覆盖 RL 节俭的更新），以及为什么 PLD 要冻结主干、RECAP 要从 anchor 重训。

熵层面：熵是 RL 的探索预算。 无干预的 RL 中策略熵早期急剧下降、探索枯竭，性能与熵满足经验定律 $R \approx -ae^H + b$ ——性能是“用熵换来的”，天花板在熵耗尽前可预测。**在连续动作空间，熵坍塌更危险：**它等于动作多样性丧失、mode collapse 到单一抓取姿态。这正是 VLA RL 必须显式管熵（有界 mode-seeking 目标、避开 Forward-KL 熵爆炸与 Hard-CE 过早坍塌）的原因，也呼应了 PLD 残差 actor 保留 entropy 正则、RECAP 用 CFG 阈值而非大 β （大 β 把动作推到支撑边缘）的设计。

一句话收束：VLA 真机后训练应是「SFT 求覆盖（求准）→ 在 OOD 峰附近切 RL 求锐化与修复（求泛化），全程显式管熵、用 off-policy replay 缓解 rollout 成本」——PLD 与 RECAP 是这一原则下两种成功的工程实例化，而它们共同留下的经验数据层空白，正是 HEARS-Buffer 的切入点。

9 关键文献

主线方法：PLD (Xiao et al., Self-Improving VLA with Data Generation via Residual RL, ICLR 2026, arXiv:2511.00091)；RECAP / $\pi^*0.6$ (Physical Intelligence, $\pi^*0.6$: a VLA That Learns From Experience, arXiv:2511.14759, 未开源)。

支撑机制：Cal-QL (Nakamoto et al., NeurIPS 2023, arXiv:2303.05479)；RLPD (Ball et al., ICML 2023, arXiv:2302.02948)；CQL (Kumar et al., NeurIPS 2020)；AWAC (Nair et al., arXiv:2006.09359)；IQL (Kostrikov et al., arXiv:2110.06169)；Classifier-Free Guidance (Ho & Salimans, 2022)；CFGRL (Classifier-Free Guidance for RL)。

训练动力学（完整列表见 docs/01_训练动力学机制解读.md）：SFT Memorizes RL Generalizes (arXiv:2501.17161)；RL Heals OOD Forgetting (arXiv:2509.12235)；Entropy Mechanism of RL (arXiv:2505.22617)；DFT (arXiv:2508.05629)；UPGE/HPT (arXiv:2509.04419)；MIFO (arXiv:2510.04454)。

说明：标“未开源”者其精确常数（如 RECAP 的 C_{fail} 、最终 checkpoint）论文未给；arXiv 编号中 26xx.xxxxx 为整理时最新预印本，引用前请核对实际发表信息。残差参数量（ $_{10M}$ ）、样本复用倍率（ $250\times$ ）为精读推断的量级表述。